MINI PROJECT VISUAL PROGRAMMING

PENJADWALAN AKTIVITAS

Disusun Oleh:

Valerine Jesika Dewi

F1D02310027

Pemrograman Visual C

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MATARAM

2026

A. Deskripsi Aplikasi

Aplikasi Penjadwal Aktivitas adalah solusi perangkat lunak manajemen waktu berbasis desktop yang dirancang khusus untuk meningkatkan produktivitas individu melalui pengorganisasian jadwal yang terstruktur. Secara teknis, aplikasi ini dibangun di atas bahasa pemrograman Python dengan memanfaatkan PySide6 sebagai *framework* antarmuka grafisnya, yang merupakan *binding* Python resmi untuk Qt. Penggunaan framework ini memungkinkan aplikasi memiliki performa yang stabil serta tampilan yang responsif pada platform desktop.

1. Arsitektur dan Manajemen Data

Aplikasi ini mengadopsi prinsip Separation of Concerns (SoC), di mana setiap fungsionalitas dipisahkan ke dalam modul-modul khusus untuk mempermudah pemeliharaan dan pengembangan di masa depan:

a. **Persistensi Data**: Untuk penyimpanan data jangka panjang, aplikasi menggunakan mesin database SQLite. Database ini bersifat *serverless* dan disimpan secara lokal dalam file penjadwal.db. Tabel utama dalam database dirancang untuk menyimpan metadata lengkap, mulai dari judul aktivitas, koordinat waktu (tanggal, jam mulai, jam selesai), hingga catatan deskriptif dan lokasi.

b. **Logika Kontroler**: Seluruh aliran data diatur melalui berkas logic.py yang bertindak sebagai jembatan antara antarmuka (UI) dan database. Di sini, dilakukan validasi ketat, seperti memastikan judul tidak lebih dari 100 karakter dan mencegah adanya jadwal di mana waktu selesai mendahului waktu mulai.

2. Desain Antarmuka dan Pengalaman Pengguna (UX/UI)

Visualisasi aplikasi diarahkan pada kenyamanan pengguna dengan tema Modern Dark Mode. Penggunaan berkas CSS eksternal (style.qss) memungkinkan pemisahan antara logika program dan estetika visual.

a. **Indikator Visual**: Aplikasi memberikan *feedback* visual instan melalui pewarnaan teks pada tabel berdasarkan tingkat prioritas. Sebagai contoh, aktivitas dengan prioritas "Tinggi" akan berwarna merah (#ed4245), sedangkan prioritas "Rendah" berwarna hijau (#3ba55c).

b. **Dashboard Statistik**: Pada bagian atas antarmuka utama, terdapat empat kartu statistik dinamis (*Stat Cards*) yang memberikan ringkasan kuantitatif mengenai total aktivitas, jumlah tugas yang telah selesai, tugas yang masih tertunda, serta jumlah agenda khusus untuk hari yang sedang berjalan.

3. Mekanisme Kerja Fitur Utama

Aplikasi menyediakan fungsionalitas manajemen data yang komprehensif atau dikenal sebagai operasi CRUD:

a. **Penambahan dan Pengeditan**: Pengguna berinteraksi dengan DialogAktivitas (di dalam ui_dialog.py) yang menyediakan form input lengkap dengan *widget* kalender (*QDateEdit*) dan pemilih waktu (*QTimeEdit*) yang memudahkan input data temporal secara akurat.

b. **Sistem Filtrasi Canggih**: Salah satu keunggulan aplikasi ini adalah kemampuannya melakukan filter data secara *multi-parameter*. Pengguna dapat mencari aktivitas berdasarkan kombinasi tanggal tertentu, kategori (seperti Kuliah, Tugas, atau Organisasi), dan status penyelesaian secara sekaligus melalui *toolbar* pencarian.

c. **Otomatisasi Status**: Terdapat fitur "Tandai Selesai" yang memungkinkan pengguna mengubah status aktivitas hanya dengan satu klik. Perubahan ini secara otomatis memicu pembaruan pada database dan dashboard statistik, memberikan perasaan pencapaian (*sense of achievement*) bagi pengguna dalam mengelola tugasnya.

Secara keseluruhan, aplikasi ini bukan sekadar alat pencatat biasa, melainkan sistem informasi terpadu yang menggabungkan kemudahan penggunaan dengan keamanan data lokal dan logika bisnis yang solid.

B. Tujuan Aplikasi

Berikut adalah tujuan dari pengembangan aplikasi Penjadwal Aktivitas :

1. Mengimplementasikan sistem penyimpanan data permanen berbasis database relasional SQLite melalui file database.py untuk menjamin keamanan dan persistensi informasi jadwal pengguna.
2. Menerapkan prinsip arsitektur Separation of Concerns (SoC) dengan memisahkan tanggung jawab kode ke dalam modul database, logika bisnis (logic.py), dan antarmuka pengguna (ui_main.py serta ui_dialog.py).
3. Menyediakan fitur manajemen aktivitas yang komprehensif, mencakup pengorganisasian berdasarkan kategori, penentuan skala prioritas, dan pencatatan lokasi guna mempermudah manajemen waktu harian.
4. Menghasilkan visualisasi data statistik secara real-time untuk memantau progres aktivitas, jumlah jadwal hari ini, serta status penyelesaian tugas melalui dashboard yang interaktif. Menjamin integritas data melalui mekanisme validasi pada

AktivitasController, seperti verifikasi rentang waktu mulai-selesai dan pembatasan panjang karakter judul aktivitas.

5. Mengembangkan antarmuka pengguna (GUI) yang modern dan intuitif dengan memanfaatkan framework PySide6 serta kustomisasi estetika menggunakan Qt Style Sheets (style.qss).

C. Struktur Project dan Penerapan SoC

1. Struktur Proyek

Aplikasi ini diorganisir ke dalam beberapa berkas utama yang memiliki tanggung jawabspesifik untuk memastikan kode tetap rapi dan modular:

a. main.py: Berkas utama yang berfungsi sebagai *entry point* aplikasi, bertanggung jawab untuk inisialisasi database, memuat *stylesheet*, dan menjalankan siklus hidup aplikasi PySide6.

b. database.py: Berkas yang menangani seluruh interaksi tingkat rendah dengan SQLite, termasuk pembuatan tabel, eksekusi *query* CRUD, dan pengelolaan koneksi database.

c. logic.py: Berisi kelas AktivitasController yang berfungsi sebagai pengelola logika bisnis, validasi input pengguna, dan jembatan antara antarmuka dengan database.

d. ui_main.py: Mengelola komponen visual jendela utama (*MainWindow*), termasuk pengaturan *layout*, tabel aktivitas, *toolbar* pencarian, dan dashboard statistik.

e. ui_dialog.py: Berisi kelas dialog khusus untuk input data (DialogAktivitas) dan informasi aplikasi (DialogTentang) guna memisahkan formulir dari jendela utama.

f. style.qss: Berkas *Qt Style Sheets* yang menyimpan seluruh aturan desain visual, warna, dan tema *dark mode* aplikasi secara terpisah dari logika program.

g. penjadwal.db: Berkas database lokal yang dihasilkan secara otomatis untuk menyimpan seluruh data aktivitas secara persisten.

2. Penerapan Separation of Concerns (SoC)

Penerapan SoC dalam proyek ini bertujuan untuk memisahkan aplikasi ke dalam bagian-bagian yang tidak saling tumpang tindih fungsinya:

a. Pemisahan Lapisan Data (Data Layer): Seluruh kode SQL dan manajemen koneksi dipusatkan di database.py, sehingga bagian lain dari aplikasi tidak perlu mengetahui kerumitan struktur tabel database.

b. Pemisahan Logika Bisnis (Logic Layer): Aturan validasi (seperti memastikan waktu selesai tidak lebih awal dari waktu mulai) diisolasi dalam

AktivitasController di logic.py, sehingga logika ini tetap konsisten baik saat menambah maupun mengedit data.

c. Pemisahan Antarmuka (Presentation Layer): Komponen GUI didefinisikan secara terpisah di ui_main.py dan ui_dialog.py, yang hanya fokus pada cara menampilkan informasi dan menangani kejadian klik dari pengguna.

d. Pemisahan Estetika (Style Layer): Dengan menggunakan file style.qss, perubahan desain visual seperti warna tombol atau font dapat dilakukan tanpa menyentuh kode Python sama sekali.

e. Pemisahan Alur Inisialisasi: Proses startup aplikasi seperti pengecekan file database dan pemuatan tema dipisahkan dalam fungsi mandiri di main.py agar alur eksekusi aplikasi menjadi lebih jelas dan teratur.

D. Desain Database

Sistem ini menggunakan SQLite, sebuah *database engine* relasional yang bersifat *serverless* dan *self-contained*. Data disimpan secara lokal dalam satu berkas bernama `penjadwal.db` yang dikelola melalui fungsi-fungsi di dalam `database.py`. Penggunaan `conn.row_factory = sqlite3.Row` memungkinkan data hasil *query* diakses layaknya kamus (*dictionary*), yang mempermudah pemetaan data ke dalam komponen antarmuka.

Database terdiri dari satu tabel utama bernama `aktivitas` yang dirancang untuk menyimpan seluruh detail jadwal. Berikut adalah spesifikasi kolom-kolomnya:

a. id: Bertipe INTEGER yang berfungsi sebagai kunci utama (Primary Key) dan akan bertambah secara otomatis (Auto-increment) untuk setiap entri baru.

b. judul: Bertipe TEXT yang menyimpan nama aktivitas, wajib diisi (Not Null), dan dibatasi maksimal 100 karakter pada lapisan logika.

c. tanggal: Bertipe TEXT yang menyimpan tanggal aktivitas dalam format YYYY-MM-DD dan wajib diisi.

d. waktu_mulai: Bertipe TEXT yang menyimpan jam dimulainya aktivitas dalam format HH:mm dan wajib diisi.

e. waktu_selesai: Bertipe TEXT yang menyimpan jam berakhirnya aktivitas dalam format HH:mm dan wajib diisi.

f. kategori: Bertipe TEXT untuk mengelompokkan jenis aktivitas seperti Kuliah, Tugas, atau Organisasi dan wajib diisi.

g. prioritas: Bertipe TEXT untuk menentukan tingkat urgensi antara Tinggi, Sedang, atau Rendah dan wajib diisi.

h. lokasi: Bertipe TEXT untuk menyimpan informasi tempat atau tautan pertemuan yang bersifat opsional.

i. deskripsi: Bertipe TEXT untuk menyimpan catatan tambahan atau rincian aktivitas yang bersifat opsional.
j. status: Bertipe TEXT untuk melacak progres (Belum Selesai, Sedang Berlangsung, Selesai) dengan nilai bawaan 'Belum Selesai'.
k. dibuat_pada: Bertipe TEXT yang secara otomatis mencatat waktu pembuatan data menggunakan fungsi internal SQLite datetime('now','localtime').

Desain database ini didukung oleh beberapa fungsi operasional di dalam database.py untuk menjamin integritas data:

a. init_db(): Menjalankan perintah CREATE TABLE IF NOT EXISTS untuk memastikan struktur tabel tersedia saat aplikasi pertama kali dijalankan.
b. tambah_aktivitas(data): Menggunakan named placeholders (:judul, :tanggal, dst.) untuk mencegah serangan SQL Injection saat menyimpan data baru.
c. ambil_semua_aktivitas(): Mendukung pencarian dinamis dengan klausa WHERE 1=1 yang memungkinkan penambahan filter (tanggal, kategori, status) secara fleksibel melalui query SQL.
d. update_status(id, status): Fungsi khusus untuk memperbarui kolom status secara cepat, yang digunakan oleh fitur "Tandai Selesai" pada antarmuka.
e. ambil_statistik(): Melakukan agregasi data menggunakan fungsi COUNT(*) untuk menghitung total tugas, tugas selesai, dan jadwal hari ini secara efisien langsung dari sisi database.

E. Fitur Utama dan Cara Kerja

1. Fitur Utama Aplikasi

a. Manajemen CRUD Lengkap

Pengguna dapat menambah, melihat, mengubah, dan menghapus aktivitas melalui antarmuka yang terintegrasi dengan database SQLite.

b. Sistem Filtrasi Multi-Parameter

Menyediakan fitur pencarian jadwal yang fleksibel berdasarkan rentang tanggal, kategori tertentu, hingga status penyelesaian.

c. Dashboard Statistik Real-Time

Menampilkan ringkasan data berupa total aktivitas, jumlah agenda hari ini, serta status tugas yang sudah selesai atau masih tertunda melalui kartu statistik di bagian atas aplikasi.

d. Validasi Input Otomatis

Menjamin kualitas data dengan melakukan pengecekan judul (maksimal 100 karakter) dan validasi waktu agar jam selesai tidak mendahului jam mulai.

e. Indikator Visual Prioritas dan Status

Menggunakan sistem pewarnaan pada tabel untuk membedakan urgensi tugas (Tinggi, Sedang, Rendah) dan status progres (Selesai, Sedang Berlangsung, Belum Selesai).

f. Antarmuka Modern (Dark Mode)

Mengimplementasikan desain visual yang elegan dan nyaman di mata menggunakan Qt Style Sheets (QSS) eksternal.

2. Mekanisme Cara Kerja

a. Proses Inisialisasi

Saat main.py dijalankan, aplikasi memanggil init_db() untuk memastikan tabel aktivitas sudah tersedia di database lokal dan memuat file style.qss untuk mengatur tampilan.

b. Alur Penambahan Data

Pengguna mengisi formulir pada DialogAktivitas di ui_dialog.py. Data tersebut dikirim ke AktivitasController di logic.py untuk divalidasi. Jika valid, controller memanggil fungsi tambah_aktivitas di database.py untuk menyimpannya secara permanen.

c. Mekanisme Pengolahan Data (Logic Layer)

AktivitasController bertindak sebagai perantara yang membersihkan input (seperti menghapus spasi berlebih dengan .strip()) sebelum data diproses oleh database untuk menjaga konsistensi informasi.

d. Sistem Penampilan Tabel

Fungsi _muat_data pada MainWindow mengambil data dari database, lalu melakukan iterasi untuk memasukkannya ke dalam QTableWidget. Di sini, logika pewarnaan diterapkan berdasarkan nilai prioritas dan status yang diambil.

e. Pemutakhiran Statistik

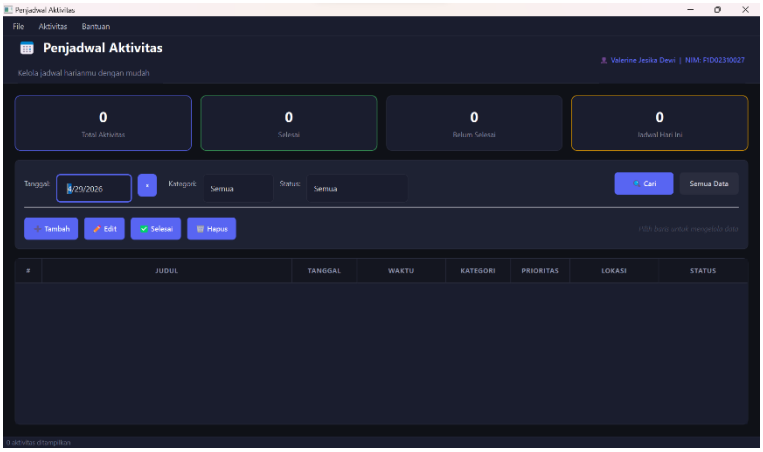
Setiap kali terjadi perubahan data (tambah, edit, atau hapus), aplikasi memanggil ambil_statistik() di database.py yang menjalankan perintah SQL COUNT(*) untuk memperbarui angka pada dashboard secara instan.

f. Fitur Tandai Selesai

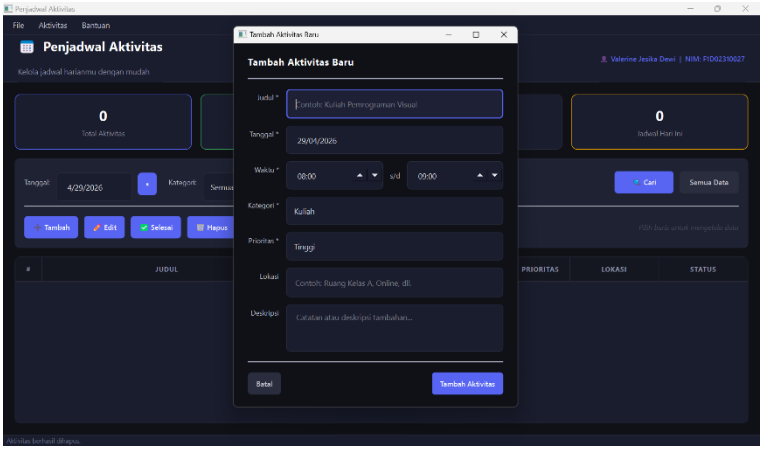
Melalui interaksi pada tabel, pengguna dapat memicu fungsi update_status yang secara spesifik hanya mengubah kolom status tanpa mengganggu metadata aktivitas lainnya.

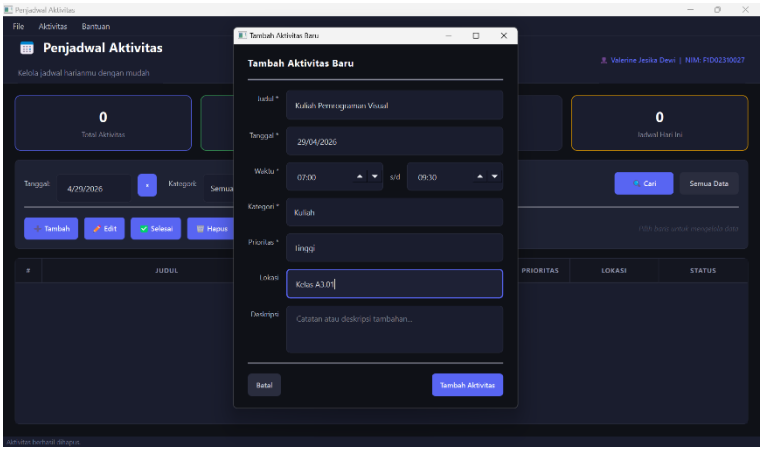
F. Output

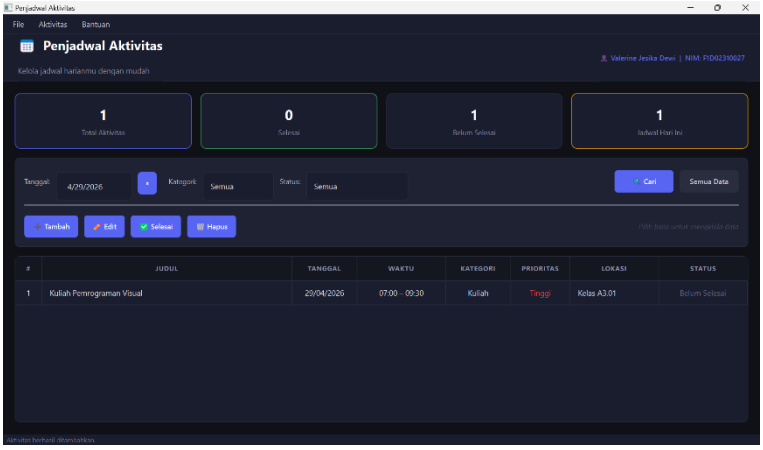
1. Tampilan Awal

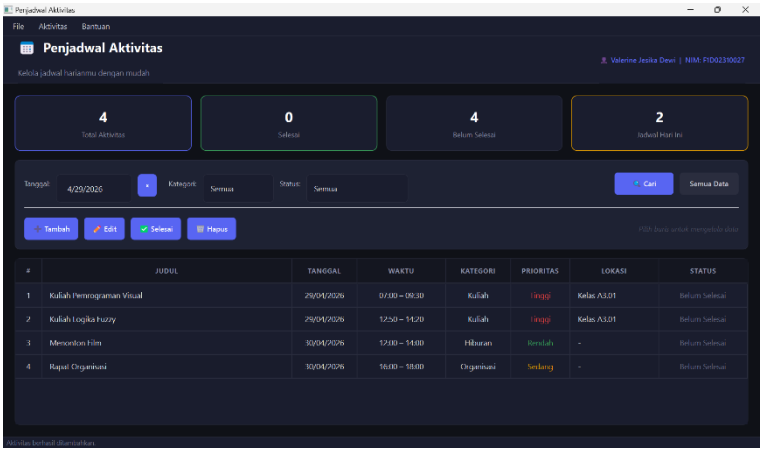


2. Tambah Aktivitas

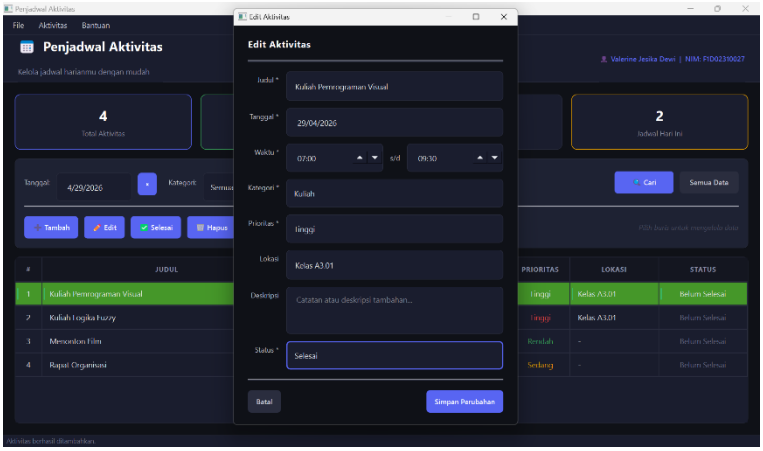


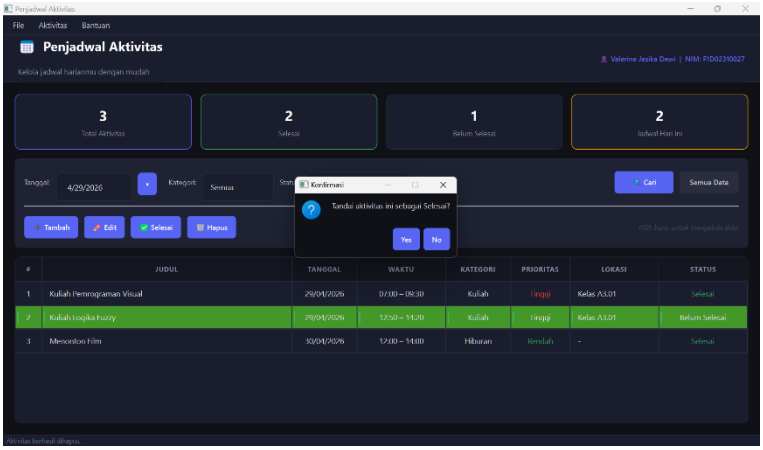


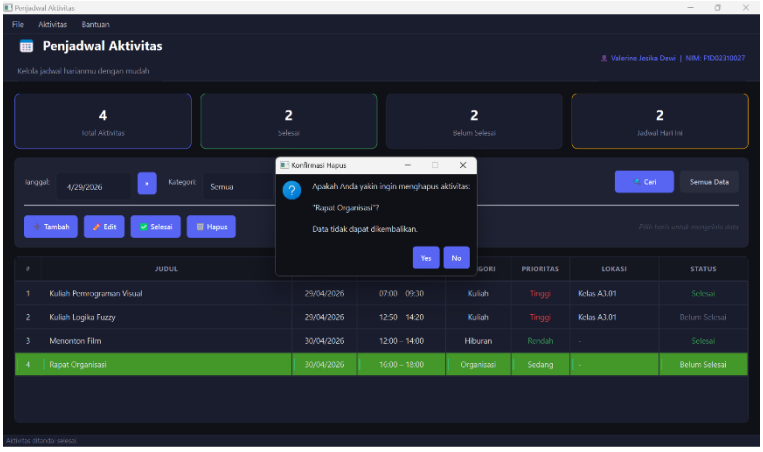




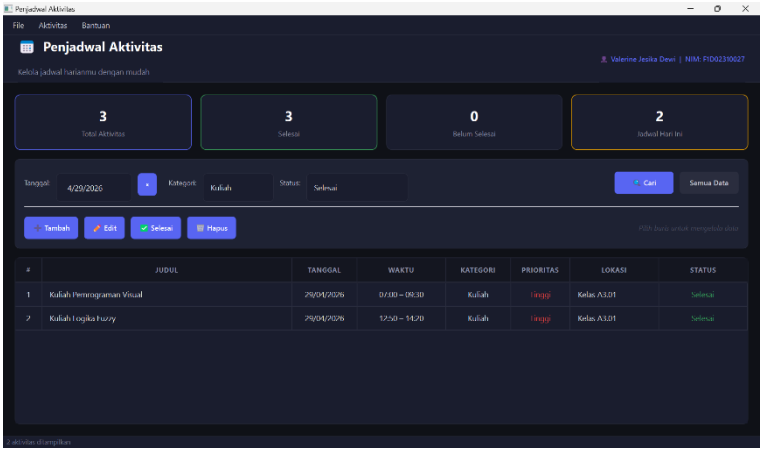
3. Tampilan Edit, Selesai, dan Hapus

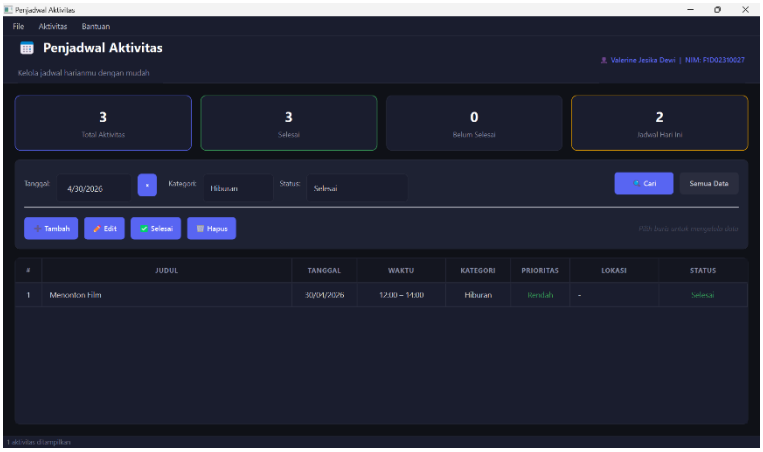






4. Tampilan Filter Pencarian





G. Link Youtube dan Github

Github : https://github.com/Valerinejsc/pv26-miniproject-PenjadwalanAktivitas-F1D02310027